

Глава 8. Транспортный уровень

В данной главе будут рассмотрены:

- Протоколы, используемые на транспортном уровне;
- Функции и задачи данных протоколов;
- Установление сеанса и надежная передача данных с использованием протокола TCP;
- Ускоренная передача данных через протокол UDP.

8.1. Протоколы транспортного уровня

Основными протоколами транспортного уровня являются TCP и UDP. Они предназначены для передачи данных по сети, разделяя их на сегменты установленного размера. Более подробная информация о каждом протоколе будет представлена в рамках последующих тематических блоков.

8.2. Значение и функции транспортного уровня

Транспортный уровень, являющийся четвертым уровнем в модели OSI, осуществляет передачу сегментов и выполняет роль посредника между уровнем приложений и нижними уровнями, которые используются сетевыми устройствами для передачи данных.

Транспортный уровень может поддерживать несколько одновременных сеансов связи, сформированных несколькими приложениями, при этом отслеживая каждый процесс отдельно.

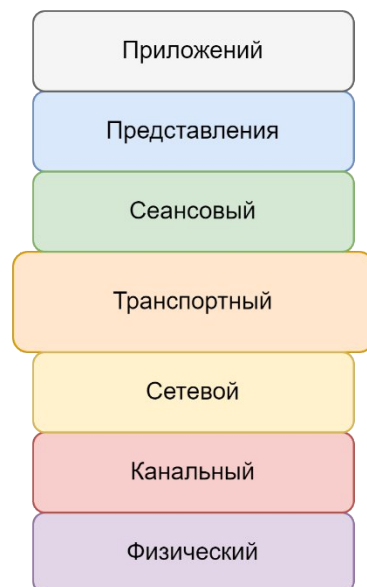


Рисунок 8.1.1. – Транспортный уровень OSI

8.2.1. Сегментация TCP/UDP

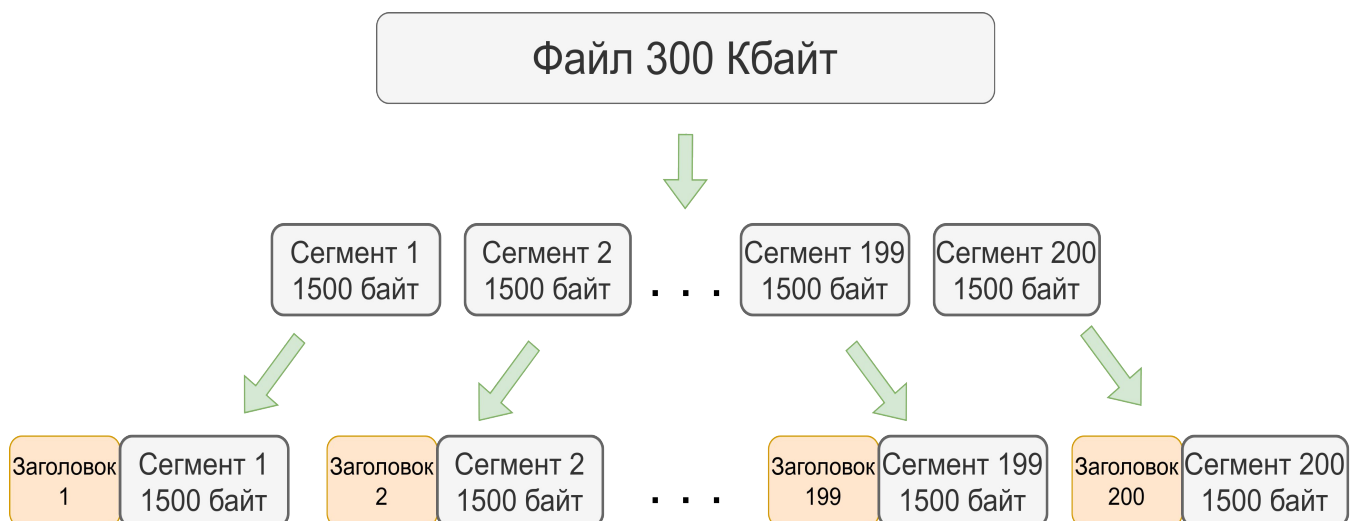


Рисунок 8.2.1.1. – Пример разбиения файла на сегменты

Некоторые приложения могут передавать большие объемы информации. Если бы данные отправлялись в виде одного большого файла, это заняло бы много времени, что могло бы привести к блокировке другого трафика по сети. Кроме того, при возникновении

каких-либо неполадок в сети, файл мог бы повредиться и требовать повторной отправки.

С целью передачи больших объемов данных существует сегментация. Суть сегментации состоит в разделении большого объема данных на меньшие фрагменты, которые затем снабжаются заголовками и передаются по сети. В большинстве случаев данные разбиваются на сегменты размером не превышающим 1460 байт. Например, при передаче файла размером 300 килобайт он будет разделен на 206 фрагментов, которые затем передадутся по сети.

Использование сегментации позволяет передавать большие объемы данных в средах передачи данных с ограниченной пропускной способностью. Кроме того, сегментация позволяет мультиплексировать данные из различных приложений через один канал передачи.

Сегментация TCP и UDP происходит по-разному. В заголовке TCP каждый сегмент имеет уникальный порядковый номер, что обеспечивает получение данных в том порядке, в котором они были отправлены. В случае UDP в заголовке сегмента отсутствует порядковый номер, поэтому данные могут быть получены в ином порядке, чем они были отправлены.

8.2.2. Мультиплексирование

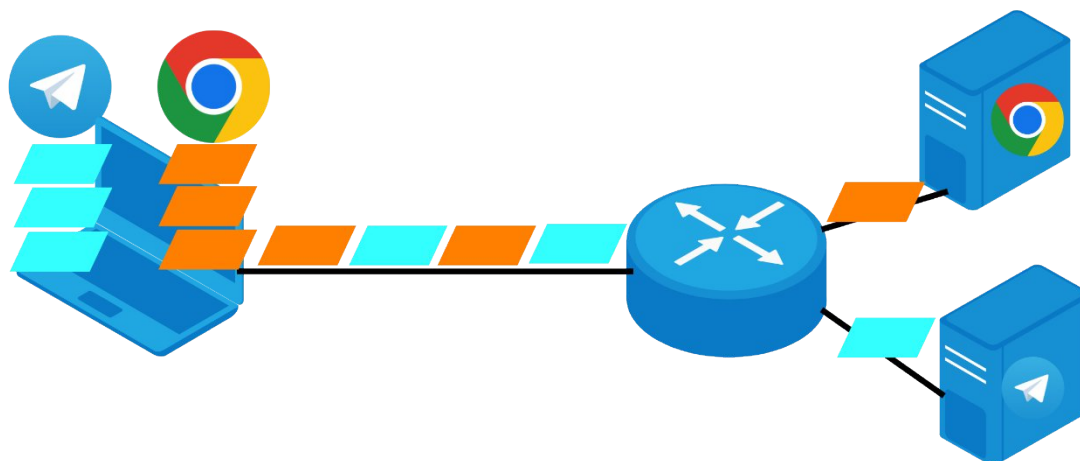


Рисунок 8.2.2.1. – Мультиплексирование

Чтобы предотвратить заполнение всей полосы пропускания одним типом данных, что может привести к блокированию передачи другой информации, используется сегментация. С помощью мультиплексирования (чередования) сегментов существует возможность передачи большого количества различных данных в одной и той же сети.

Чтобы однозначно идентифицировать созданные сегменты, на транспортном уровне добавляются специальные заголовки, состоящие из нескольких полей в двоичном виде.

Кроме того, для корректной передачи данных транспортный уровень должен однозначно идентифицировать целевое приложение. Для этого используются уникальные для каждого узла номера портов, которые присваиваются каждому приложению.

8.3. Адресация транспортного уровня

8.3.1. Адресация портов TCP и UDP

Как упоминалось ранее, идентификация приложений на транспортном уровне осуществляется с использованием портов. Введем четкое определение:

Порт – это идентификатор приложения (протокола), которое обрабатывает текущий сегмент на устройстве отправителя и на устройстве получателя.

Порты являются дополнительным компонентом IP-адреса и представляют собой значения в диапазоне от 0 до 65535. Они указываются в заголовке TCP/UDP и состоят из 16-битных чисел, отвечающих за порт источника и порт назначения, соответственно. Номер порта представлен вместе с IP-адресом в следующем формате: 192.168.1.19:80. Чтобы лучше понять значение портов, рассмотрим пример.

Предположим, есть сервер, на котором расположен сайт, почтовый клиент и другие приложения. При обращении к этому серверу, помимо IP-адреса, указывается порт непосредственно самого приложения, к которому хочет обратиться пользователь, например, 80 (HTTP). Как итог: по одному IP-адресу пользователю доступны различные приложения.

Порт источника генерируется отправителем динамически для идентификации сеанса связи между двумя приложениями, что позволяет организовывать сразу несколько сеансов связи, допустим, сразу несколько запросов HTTP.

Порт назначения также указывается отправителем, но уже представляет собой конкретное число, идентифицирующее интересующее пользователя приложение, например, запрос HTTP, так как сервер может одновременно предоставлять веб-службы (порт 80) и службу FTP (порт 21).

В таблице 8.3.1.1 показаны наиболее известные порты и приложения, которые они идентифицируют.

Таблица 8.3.1.1. Основные приложения и порты

Сервис	Порт
HTTP	80
SSH	22
DNS	53
POP3	110
IMAP	143
FTP (команды)	21
FTP (передача)	20
TFTP	69
Telnet	23
HTTPS	443
NTP	123
DHCP (клиент)	67
DHCP (сервер)	68
SMTP	25

8.3.2. Сокеты и принцип адресации

Сокет – это сочетание IP-адреса источника с номером порта источника и IP-адреса назначения с номером порта назначения. Такая связка «IP-адрес:Порт» позволяет однозначно определить конечную точку доставки сообщения, потому что оно будет адресовано не просто какому-то устройству, а конкретному приложению на данном устройстве.

Сокеты предоставляют возможность различать процессы, которые могут выполняться у пользователя, а также идентифицировать различные подключения к серверу.

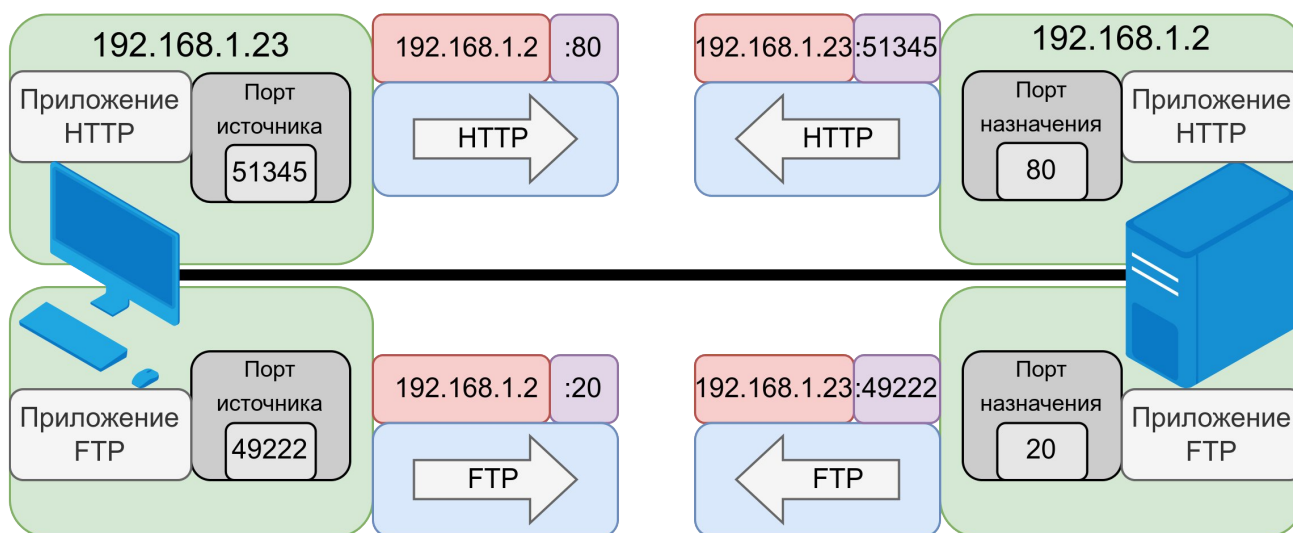


Рисунок 8.3.2.1. – Сокеты и принцип адресации

Разберем подробнее, как происходит адресация в системах с использованием сокетов. Для примера возьмем рисунок под номером 8.3.2.1. У нас есть компьютер и сервер, между которыми установлена связь по кабелю. На каждом из этих узлов установлены HTTP и FTP приложения. Допустим, компьютер хочет запросить какой-либо веб-ресурс от сервера. Он посылает соответствующий запрос, в котором указывается IP-адрес IP-сервера – 192.168.1.2 – и необходимый **порт назначения**, в нашем случае 80 (так как он стандартный для HTTP). Таким образом, мы адресуем информацию конкретному приложению. Затем сервер формирует ответ и отправляет его на IP-адрес компьютера – 192.168.1.23 – указывая соответствующий **порт источника** – 51345. Благодаря этому, ответ от сервера приходит конкретному HTTP приложению. И эта система адресации была бы не нужна, если бы на устройствах было одно единственное приложение, которое общалось исключительно между собой на устройствах.

Но приложений на устройствах гораздо больше, и на нашем рисунке мы видим еще одно приложение – FTP. Когда возникает необходимость запросить какие-то данные с сервера, FTP-приложение посылает запрос на IP-адрес сервера – 192.168.1.2 и, благодаря указанному **порту назначения** 20, информация для FTP-приложения не попадет в приложение HTTP с **портом назначения** 80. Ответ формируется по такому же принципу: указывается IP-адрес компьютера и **порт источника** 49222.

Благодаря такой системе адресации информация не теряется и не перемешивается между многочисленными приложениями на устройстве.

Использование одинаковых портов одного и того же протокола транспортного уровня двумя разными приложениями на одном сервере – недопустимо.

Порт, используемый активным приложением, считается открытым. Это означает, что транспортный уровень готов принимать сегменты, направленные на данный порт. На сервере может быть открыто множество портов, каждый из которых связан со своим определённым сервисом.

8.3.3. Группы номеров портов¹

Таблица 8.3.3.1 Группы номеров портов

Номера портов	Описание
0 - 1023	Известные порты
1024 - 49151	Зарегистрированные порты
49152 - 65535	Динамические/частные порты

Выделением и регистрацией номеров портов занимается организация IANA (Internet Assigned Numbers Authority). Всего существует три группы портов: общеизвестные (0 – 1023), зарегистрированные (1024 – 49151) и динамические/частные (49152 – 65535).

Общеизвестные порты зарезервированы для определенных служб и приложений. Они используются, например, веб-службами (HTTP – порт 80) или почтовыми клиентами (POP3 – порт 110). Так как эти порты известны большинству, приложения клиентов можно настроить так, чтобы они производили подключение к определенному сервису по конкретному порту.

Зарегистрированные порты регистрируются организацией IANA для каких-либо нестандартных служб, в основном это службы, которыми пользуются не все пользователи, к примеру, RADIUS-сервер использует порт 1812 для аутентификации пользователей.

Динамические/частные порты присваиваются, когда пользователь производит инициализации сеанса связи при подключении к определенному сервису. Такие порты генерируются в заданном диапазоне случайным образом, когда приложению требуется сетевое соединение. Сгенерированный порт используется для обмена данными с другим приложением и по завершении обмена освобождается, становясь доступным для использования другими приложениями на устройстве.

8.4. Протокол TCP

8.4.1. Функции протокола TCP

TCP – один из протоколов транспортного уровня. Его главное отличие – обеспечение надежности доставки данных. Для осуществления гарантированной доставки протокол использует следующие функции:

- Отслеживание количества сегментов, отправленных на определенный узел;
- Подтверждение получения данных;
- Если получение данных не подтвердилось, инициирование повторной передачи сегментов.

Основные задачи протокола TCP:

- Доставка сегментов в исходном порядке. Благодаря использованию нумерации сегментов и их упорядочиванию, на узле приема они собираются в правильном порядке, вне зависимости от выбранного маршрута;

¹ <https://www.iana.org/assignments/service-names-port-numbers/service-names-port-numbers.xhtml> – ссылка на группы номеров портов

- Установление сеанса связи. Перед пересылкой трафика протокол устанавливает соединение между узлом передачи и узлом приема. Это позволяет контролировать передачу данных и согласовывать объем трафика;
- Регулирование потока передачи данных. При активном использовании вычислительных мощностей сетевых узлов протокол TCP может скорректировать количество передаваемой информации. Данная функция позволяет предотвратить перегрузку узлов, а также повторную передачу данных.

Каждый отправляемый сегмент данных содержит дополнительно от 20 байт (160 бит) до 60 байт (480 бит) в заголовке; рассмотрим их подробнее.

8.4.2. Заголовок TCP-сегмента

Порт источника (16 бит)			Порт назначения (16 бит)		
Порядковый номер (32 бита)					
Номер подтверждения (32 бита)					
Длина заголовка (4 бита)	Заразervi- рованно (6 бит)	Управляющие биты (6 бит)	Размер окна (16 бит)		
Контрольная сумма (16 бит)			Срочность (16 бит)		
Опции (32 бита)					
Данные					

Рисунок 8.4.2.1. – Заголовок TCP-сегмента

Заголовок TCP-сегмента содержит от 20 до 60 байт:

1. **Порт источника и порт назначения** (по 16 бит) определяют порты отправителя и получателя.
2. **Порядковый номер** (32 бита) в сегменте используется для контроля порядка сегментов.
3. **Номер подтверждения** (32 бита) необходим для подтверждения успешного получения узлом определенного сегмента.
4. **Длина заголовка** (смещение данных) (4 бита) – длина заголовка сегмента TCP в «словах», то есть в группах по 4 байта (32 бита).
5. **Зарезервировано** (6 бит) – это биты, которые могут быть использованы в

будущем.

6. **Управляющие биты** (6 бит) содержат флаги, определяющие назначение и функции сегмента TCP.
7. **Размер окна** (16 бит) указывает количество байт данных, которые получатель может принять в данный момент.
8. **Контрольная сумма** (16 бит) используется для проверки наличия ошибок в сегменте.
9. **Срочность** (16 бит) определяет данные, которые должны быть переданы сразу.

Поле управляющих битов

Для управления сеансом связи в заголовке протокола TCP используется поле длиной 6 бит. Данные биты называются флагами.

```

Flags: 0x002 (SYN)
 000. .... = Reserved: Not set
 ...0 .... = Nonce: Not set
 .... 0... = Congestion Window Reduced (CWR): Not set
 .... .0.. = ECN-Echo: Not set
 .... ..0. = Urgent: Not set
 .... ...0 = Acknowledgment: Not set
 .... .... 0... = Push: Not set
 .... .... .0.. = Reset: Not set
> .... .... ..1. = Syn: Set
 .... .... ...0 = Fin: Not set
  
```

Рисунок 8.4.2.2. – Представление управляющих флагов в Wireshark. Флаг Syn

Поле управляющих битов может иметь следующие значения:

- SYN – флаг синхронизации порядковых номеров. Когда флаг SYN установлен, в поле «порядковый номер» передается изначальный номер последовательности (Initial Sequence Number, ISN). Первый байт данных будет передан в следующем пакете с номером последовательности, равным ISN+1.
- ACK – флаг подтверждения. Данный флаг указывает на то, что в поле «номер подтверждения» указан следующий порядковый номер сегмента, который ожидается для получения на приемной стороне. Также, данный флаг подтверждает приём предыдущих сегментов.
- FIN – флаг завершения сеанса связи. Если такой флаг установлен, это означает, что отправитель сигнализирует о завершении передачи данных и не планирует отправлять какие-либо дополнительные данные в рамках текущего соединения.
- RST – флаг сброса соединения. Установка этого флага означает немедленный разрыв соединения и игнорирование всех последующих входящих данных из этого соединения.
- PSH – флаг функции *push*. Когда такой флаг установлен, это означает, что необходимо передать все данные из буфера памяти в обработку, даже если он не был заполнен. Здесь важно сделать отступление и сказать, что протокол TCP использует буферизированную передачу данных как на отправке, так и на приеме. То есть, данные перед отправкой поступают в буфер, и при его заполнении отправляются по адресу. Такой метод позволяет снизить количество служебного трафика (заголовков) в сети.

- URG – флаг указателя важности. Установка данного флага означает, что данные в пакете необходимо обработать в приоритетном порядке.

8.4.3. Установка TCP-соединения

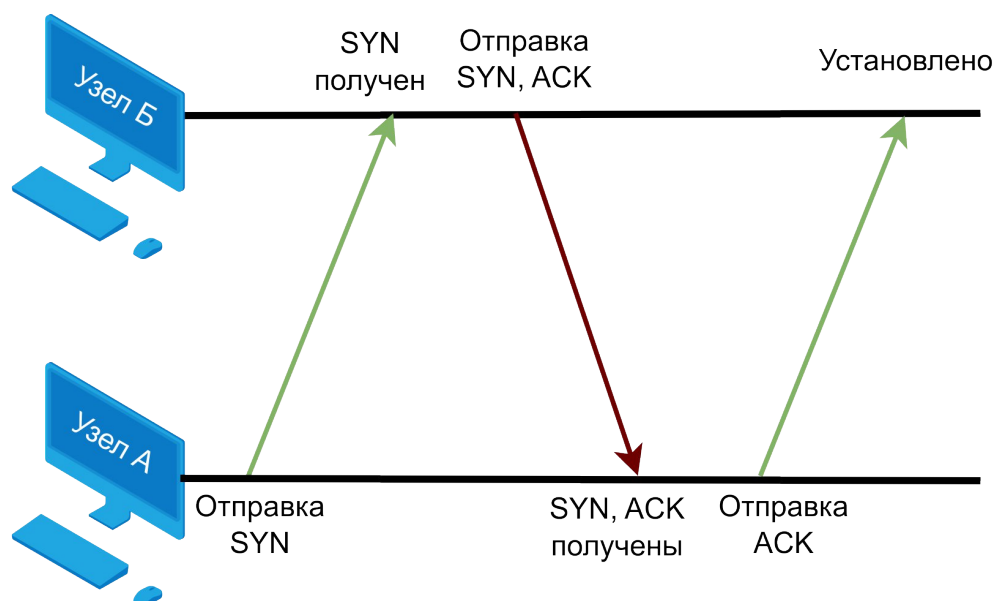


Рисунок 8.4.3.1. – Процесс установки TCP-соединения

Прежде, чем начать передачу данных, пользователь устанавливает соединение с сервером, такой процесс называется трехэтапным рукопожатием. Данный процесс согласует номера используемых портов и порядковые номера сегментов. Процесс установление TCP соединения проходит в три этапа:

- 1) Узел А, инициализирующий сессию для обмена данными «клиент-сервер», отправляет на узел Б запрос SYN;
- 2) Узел Б подтверждает запрос на инициализацию сессии для обмена данными «клиент-сервер», отправляет узлу А ответ ACK и запрос SYN «сервер-клиент»;
- 3) В конце узел А подтверждает запрос SYN «сервер-клиент» и отправляет ACK узлу Б.

Для того чтобы подробнее изучить, как происходит процесс трехэтапного рукопожатия, рассмотрим реальный пример на основе программы анализатора сетевого трафика Wireshark.

Для начала переходим в командную строку Windows, где с помощью команды `nslookup` узнаем IP-адрес веб-сайта www.eltex-co.ru, в браузере переходим на веб-страницу www.eltex-co.ru, но перед этим запускаем программу Wireshark и начинаем захватывать трафик на интерфейсе.

В фильтре Wireshark вводим команду `ip.addr == 62.109.1.166` (IP-адрес веб-сайта www.eltex-co.ru), которая покажет все захваченные пакеты, где есть адрес 62.109.1.166.

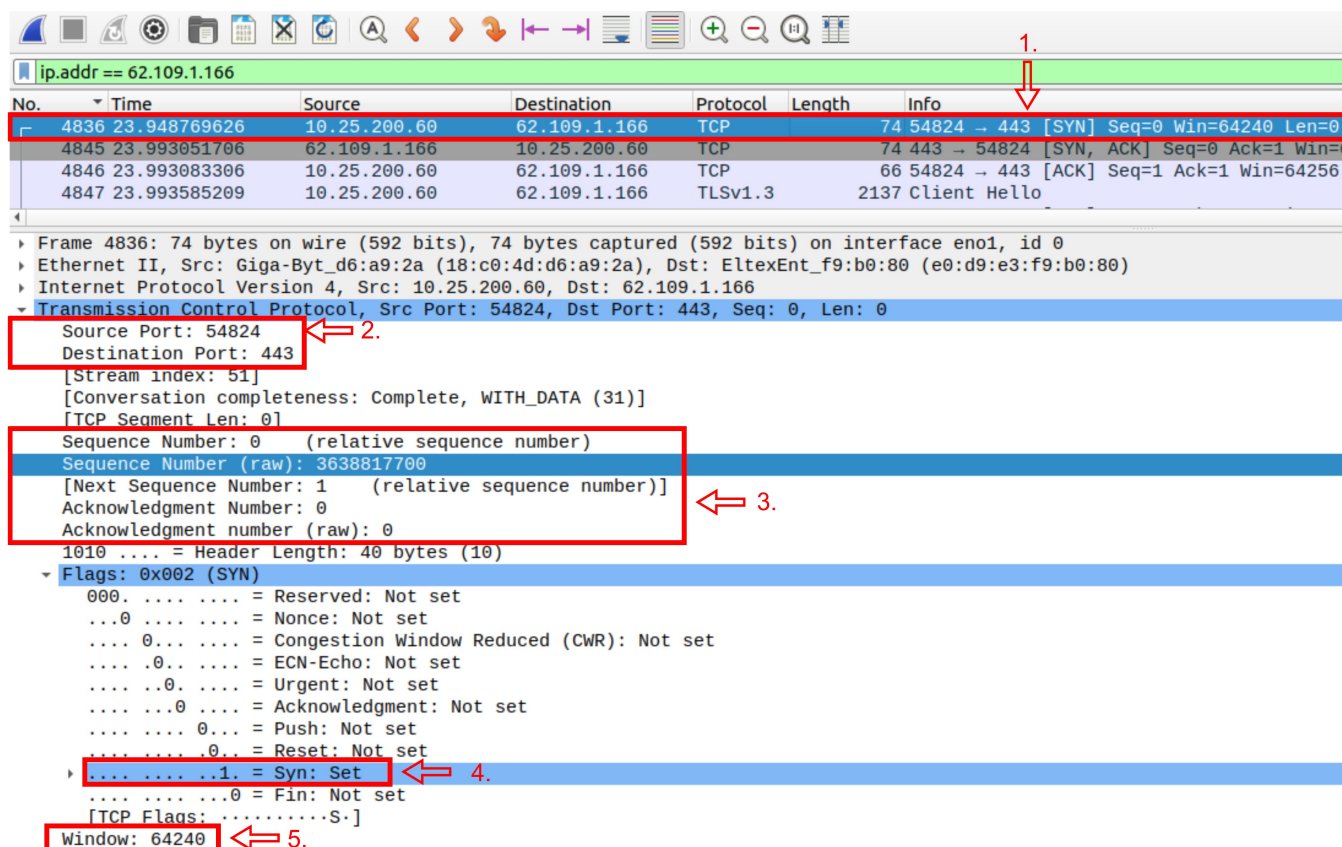


Рисунок 8.4.3.2. – Первый этап «трехэтапного рукопожатия»

Разберем подробно, что мы видим из первого этапа рукопожатия:

1. Сам пакет, который мы рассмотрим изнутри.
2. Клиент отправляет запрос на сервер, используя в качестве порта источника частный порт 54824, а в роли порта назначения – общеизвестный порт 443 (HTTPS).
3. В данном пункте необходимо внимательно разобраться. Мы видим строки:

Sequence Number: 0 – Ноль в данной строке обозначает порядковый номер относительно начала сессии. Поскольку эта последовательность является самой первой, она обозначается нулем. Важно понимать, что это значение сформировано программой, для упрощения нашего понимания начала и конца последовательности.

Sequence Number (raw): 3638817700 – Здесь представлен настоящий, «сырой» номер, присвоенный пакету, с которым он передается по сети.

Next Sequence Number: 1 – это следующий ожидаемый номер последовательности, который должен прийти. То есть, +1 к текущему, что логично.

Acknowledgment number и **Acknowledgment number (raw)** – здесь представлены номера подтверждающей последовательности (флаг ACK). Ноль потому, что ACK еще не поступил.

4. Установленный флаг SYN обозначает начало синхронизации и запрос на соединение с сервером.

5. Window 64240 – в данном случае это размер окна приема TCP. Оно обозначает максимальное количество байтов данных, которое может быть принято принимающей стороной в данный момент.

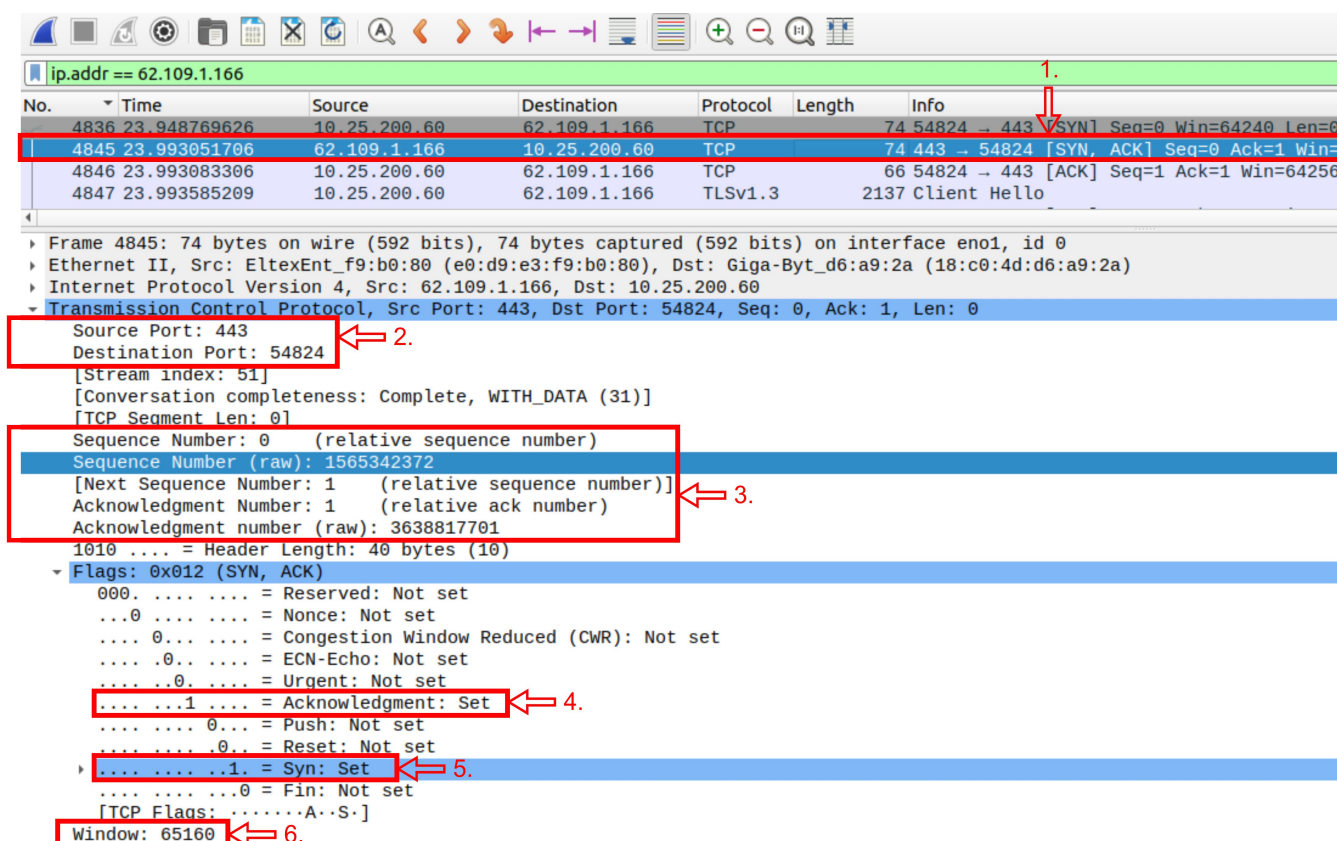


Рисунок 8.4.3.3. – Второй этап «трехэтапного рукопожатия»

Разберем подробно, что мы видим из первого этапа рукопожатия:

1. Сам пакет, который мы рассмотрим изнутри.
2. Сервер отправляет ответ на клиент, используя в качестве порта источника порт 443, а в роли порта назначения – частный порт 54824. Это происходит потому, что в ответе роли портов меняются местами.
3. В данном пункте необходимо внимательно разобраться. Мы видим строчки:

Sequence Number: 0 – Важно понимать, что здесь тоже ноль, потому что это первая последовательность со стороны сервера.

Sequence Number (raw): 1565342372 – Здесь представлен настоящий, «сырой» номер, присвоенный пакету с которым он передается по сети.

Next Sequence Number: 1 – это следующий ожидаемый номер последовательности, который должен прийти. То есть, +1 к текущему.

Acknowledgment number и **Acknowledgment number (raw)** – здесь представлены номера подтверждающей последовательности (флаг ACK). Указано **1** и **3638817701**, потому что +1 к начальному номеру последовательности.

4. Установленный флаг ACK обозначает подтверждение принятых данных.

5. Установленный флаг SYN обозначает начало синхронизации (со стороны сервера в данном случае) и запрос на соединение с клиентом.

6. Window 65160 – Как мы видим, размер окна изменился. Это связано с механизмом динамического изменения размера окна (так называемое «скользящее окно»), который будет подробно рассмотрен далее.

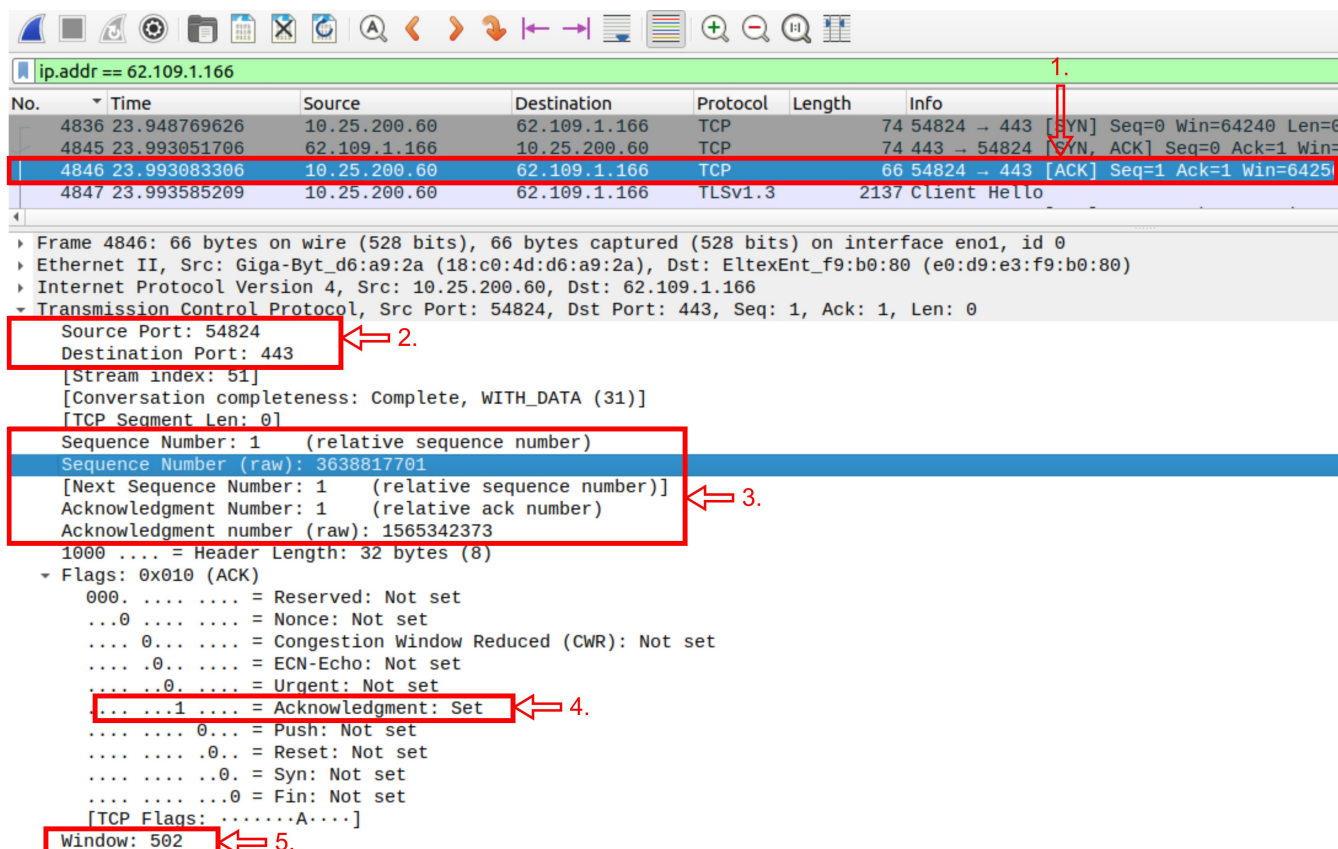


Рисунок 8.4.3.4. – Третий этап «трехэтапного рукопожатия»

Разберем подробно, что мы видим из третьего этапа рукопожатия:

1. Сам пакет, который мы рассмотрим изнутри.
2. Вновь порта поменялись местами, потому что это данные от клиента для сервера.
3. В данном пункте необходимо внимательно разобраться. Мы видим строчки:
 - Sequence Number: 1** – Последовательность сменилась на единицу.
 - Sequence Number (raw): 3638817701** – Видим закономерное +1.
 - Next Sequence Number: 1** – это следующий ожидаемый номер последовательности, который должен прийти. То есть, +1 к текущему, что логично.
 - Acknowledgment number** и **Acknowledgment number (raw)** – здесь представлены номера подтверждающей последовательности (флаг ACK). Указано **1** и **1565342373**, потому что +1 к начальному номеру последовательности.
4. Установленный флаг ACK обозначает подтверждение принятых данных.
5. Window 502 – Размер окна снова изменился. Он будет изменяться каждый раз в зависимости от параметров принимающей и передающей стороны, а так же от загрузки сети.

8.4.4. Надежность TCP

8.4.4.1. Подтверждение и размер окна

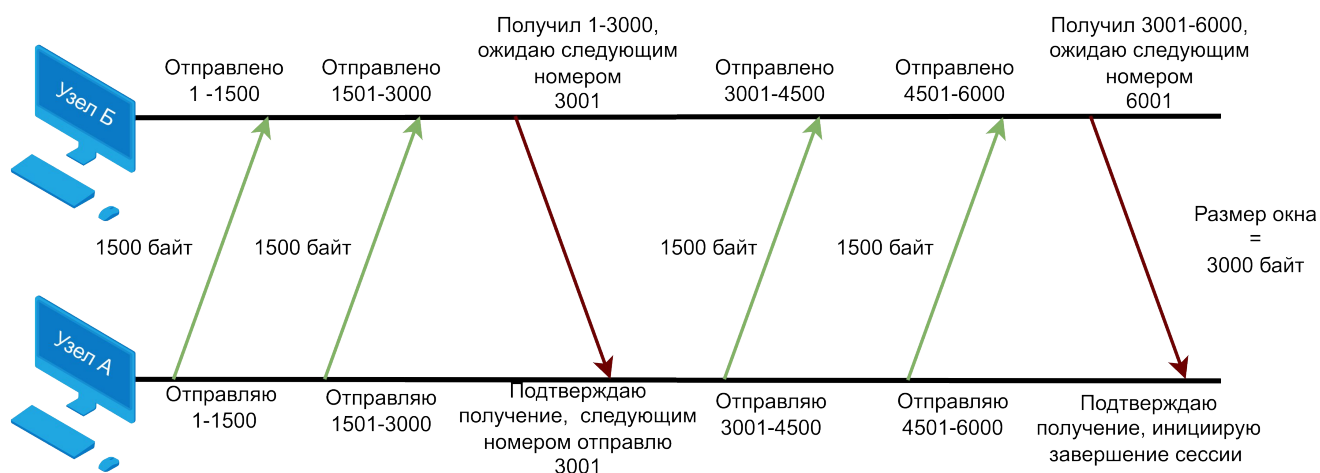


Рисунок 8.4.4.1.1. – Процесс передачи данных при TCP-соединении

В заголовке TCP-сегмента существует 16-битное поле для указания размера окна. Данный параметр необходим для управления объемом данных, который целевой узел может успешно принять и обработать.

Первичный размер окна согласуется при установлении сеанса в рамках трехэтапного рукопожатия. При получении информации о размере окна получателя, отправитель должен ограничить количество одновременно передаваемых байт.

Как правило, получатель не дожидается получения всех пакетов, прежде чем отправить подтверждение, а делает это после получения каждого 2 сегментов (количество может варьироваться). Такой процесс называется скользящее окно. За счет непрерывной передачи сегментов повышается скорость работы сети, но в случае отсутствия подтверждения с принимающей стороны необходимо будет повторно переслать эти данные и уменьшить размер окна, чтобы снизить объем передаваемого трафика. Это может помочь в пиковые моменты, когда принимающая сторона перегружена.

На рисунке 8.4.6.1.1 приведен пример передачи данных при TCP-соединении с размером окна, равным 3000 байт, который был согласован в процессе установления TCP-соединения. После передачи получателю 3000 байт, последний уведомляет отправителя об успешном приеме сообщением с флагом ACK, и отправитель начинает передавать следующий блок данных.

8.4.4.2. Потеря данных и повторная передача

В любой, даже хорошо организованной сети, обычно возникают потери данных. Чтобы справиться с этой проблемой, протокол TCP имеет механизм повторной передачи сегментов. При получении сегментов целевым узлом подтверждаются только те данные, которые поступили в непрерывной последовательности байт. Если после истечения таймаута узел-отправитель не получил подтверждение о доставке, он повторно отправляет сегменты, начиная с последнего полученного подтверждения (ACK).

К примеру, были получены сегменты с порядковыми номерами от 1 до 3000 и от 6001 до 9000, то номер ACK будет равен 3001.

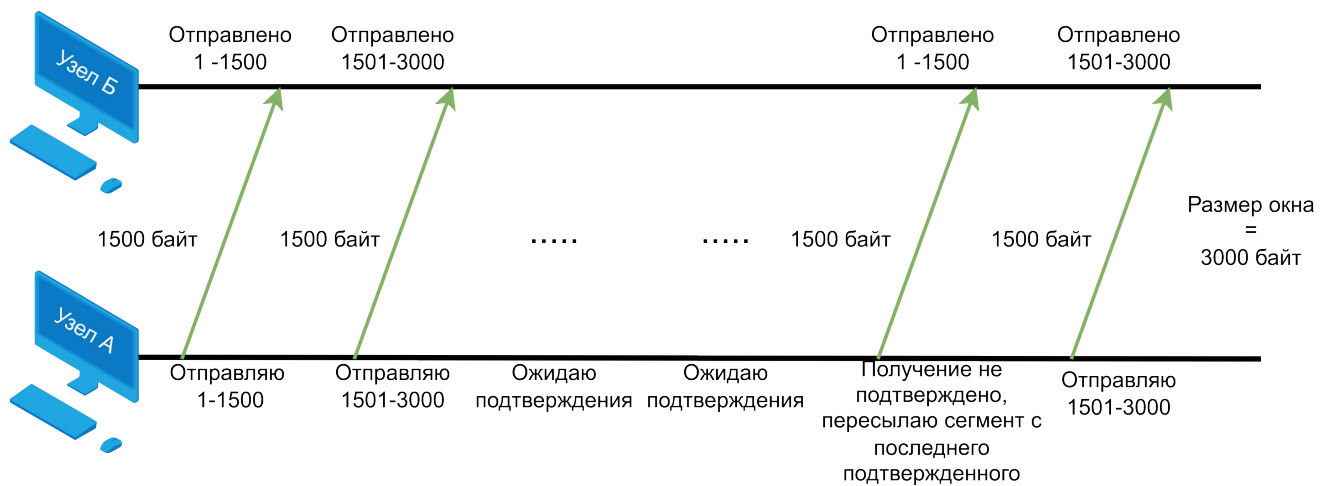


Рисунок 8.4.4.2.1. – Потеря данных и повторная передача

8.4.5. Завершение TCP-сеанса

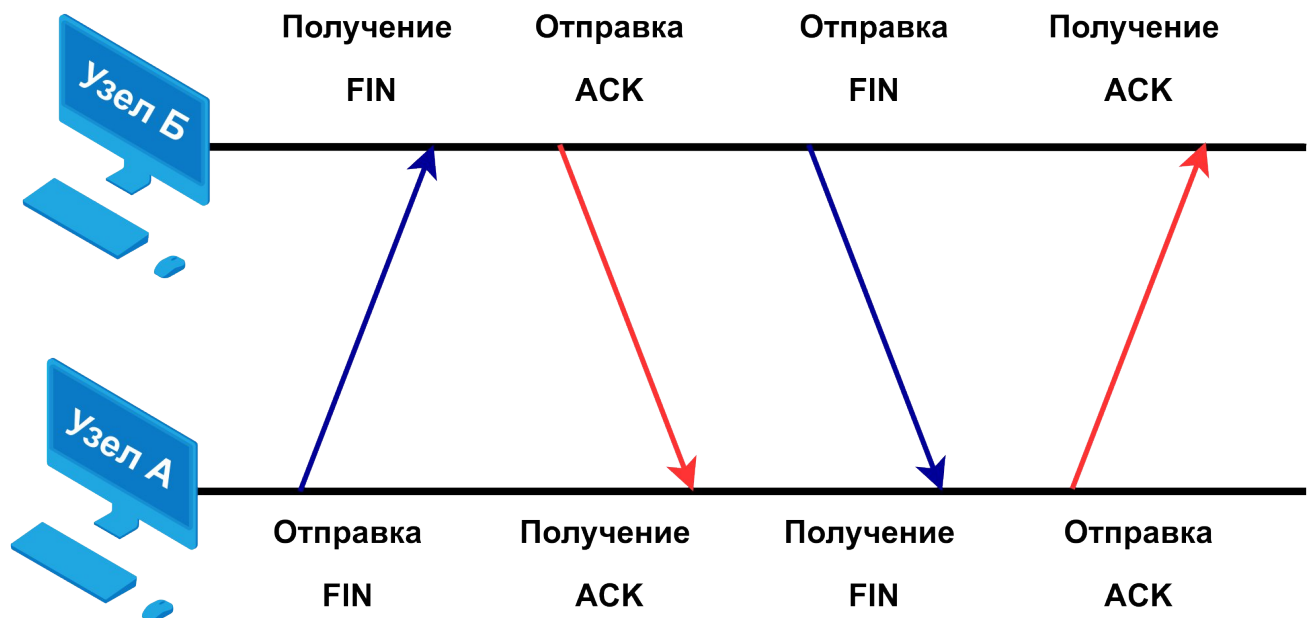


Рисунок 8.4.5.1. – Процесс завершения соединения TCP-сеанса

Для завершения сеанса передачи данных в заголовке сегмента TCP устанавливается управляющий бит флага FIN (Finish). Процесс завершения соединения сеанса связи заключается в отправке каждым узлом сегмента с флагом FIN и подтверждении ACK, итог: чтобы завершить один сеанс связи, потребуется четыре раза произвести обмен сообщениями, с помощью которых на обеих сторонах завершится сеанс связи.

- 1) После завершения отправки данных, узел А отправляет сегмент с установленным в заголовке флагом FIN для прекращения сессии «клиент-сервер»;
- 2) Узел Б принимает запрос на завершение соединения и отправляет подтверждение ACK узлу А;
- 3) Узел Б отправляет сегмент с установленным в заголовке флагом FIN для прекращения сессии «сервер-клиент». ;
- 4) Узел А, в свою очередь, отправляет ответ ACK для подтверждения получения FIN от узла Б.

Для оптимизации передаваемых сообщений флаги ACK и FIN от Узла Б могут передаваться в одном сегменте, то есть общая последовательность обмена сегментами при завершении сеанса может выглядеть так: FIN – FIN.ACK – ACK.

8.4.6. Установление TCP-соединения и управление потоком

Данные при передаче по протоколу TCP делятся на сегменты, которые при выборе различных путей маршрутизации могут быть доставлены на узел назначения в измененном порядке. Для того чтобы воссоздать исходный порядок сегментов, протокол TCP использует порядковые номера в заголовках каждого сегмента.

При установлении сеанса связи задается начальный порядковый номер сеанса (ISN), представляющий собой **случайное** начальное значения счетчика байт, которые были переданы целевому узлу. При дальнейшей передаче сегментов указанное число увеличивается на определенное число байт, что позволяет контролировать передачу данных.

Стоит обратить внимание, что ISN не является строго определенным числом, а задается **случайно**. Таким образом, можно предотвратить осуществление вредоносных атак.

На узле назначения все полученные сегменты распределяются в правильном порядке и только после этого передаются на уровень приложений.

8.5. Протокол UDP

8.5.1. Функции UDP

Рассмотренные ранее функции гарантированной доставки протокола TCP чаще всего приводят к задержкам при передаче данных. В случае, если главным приоритетом для приложения является скорость, а не гарантированная передача всех сегментов, следует отдать предпочтение протоколу UDP.

Протокол UDP так же, как и TCP, использует сегментацию данных, при этом не задействует процессы, отвечающие за уведомление об успешном получении сегмента, что позволяет значительно быстрее передавать данные. Пользуясь преимуществом в скорости и отсутствием подтверждения полученной информации, протокол UDP используется при передаче данных которые могут перенести кратковременные потери во время передачи, например, потоковой видео и звуковой информации, где потеря одной или нескольких датаграмм, т.е. с незначительными перебоями, может не повлиять на общую ситуацию в целом во время передачи.

8.5.2. Основные характеристики протокола UDP:

Основными характеристиками протокола UDP являются:

- Отсутствие установления сеанса связи;
- Отсутствие повторной отправки потерянных сегментов;
- Наличие высокоскоростного потока.

Части сообщения в UDP, передающиеся без предварительного установления канала и последующей гарантии передачи, называются датаграммами. У датаграммы есть заголовок, состоящий из 8 байт и включающий в себя порт источника (16 бит), порт назначения (16 бит), длину (16 бит) и контрольную сумму (16 бит).

8.5.3. UDP-датаграммы

Датаграмма – блок информации, передающийся с помощью протокола UDP, без гарантированной доставки на узел назначения.

При отправке по протоколу UDP данные делятся на датаграммы. Как и в случае с протоколом TCP, датаграммы могут использовать различные маршруты для достижения узла назначения, тем самым прибывая на конечную точку в хаотичном порядке и обрабатываться в том порядке в котором они пришли. Однако, протокол UDP не использует порядковые номера датаграмм, поэтому не восстанавливает их исходный порядок, а просто пересылает приложению.

Порт источника (16 бит)	Порт назначения (16 бит)
Длина (16 бит)	Контрольная сумма (16 бит)
Данные	

Рисунок 8.5.3.1. – Заголовок UDP-датаграммы

Заголовок UDP-датаграммы содержит 8 байт.

Порт источника и порт назначения определяют порт отправителя и получателя соответственно.

Длина – длина датаграммы с учетом заголовка.

Контрольная сумма обеспечивает контроль правильности данных, которые передаются в заголовке.

Заметим, в заголовке протокола UDP нет полей, которые могут использоваться для контроля пересылки и правильности «сборки» информации на стороне получателя.

Ниже приведена таблица для сравнения основных функций протоколов TCP и UDP:

Таблица 8.5.3.1. Сравнение функций протоколов TCP и UDP

Параметры	TCP	UDP
Гарантия доставки	Является надежным протоколом транспортного уровня, гарантирует доставку сообщения до узла назначения	Не обеспечивает надежную доставку сообщений
Упорядоченная доставка	Есть. Для того чтобы воссоздать исходный порядок сегментов, протокол TCP использует порядковые номера в заголовках каждого сегмента	Нет. Протокол UDP не использует порядковые номера датаграмм, поэтому не восстанавливает их исходный порядок, а просто пересылает приложению
Установка соединения	Устанавливает соединение перед отправкой данных	Не устанавливает соединение
Повторная передача данных	Повторная передача сегментов в случае потери	Нет повторной передачи
Скорость передачи	Из-за дополнительных функций гарантии доставки имеет увеличенное время передачи данных	Имеет высокую скорость передачи данных
Сферы применения	Передача сообщений электронной почты, HTML-страниц, FTP, SMTP, DNS	TFTP, DHCP, VoIP, DNS

8.5.4. Основные типы приложений, которые используют UDP:

- Приложения, которые могут самостоятельно гарантировать полную передачу данных например, TFTP. В таком случае, контроль правильности передачи данных осуществляется приложением 7-го уровня модели OSI, а не самим протоколом;
- Приложения, осуществляющие только отправку запросов и получение ответов (например, DHCP);
- Приложения для передачи мультимедийного контента в режиме реального времени (например, VoIP).